

Тема научно-исследовательской работы:
«Разработка инструментов для распараллеливания
алгоритмов»

Тема доклада:
«Прототип фреймворка для параллельного выполнения графа
задач»

Выполнил:
Студент ИИТММ гр. 3825М1ФИ1
Боровков Сергей Александрович

Научный руководитель:
к.т.н., доц. ВВСП
Козинов Евгений Александрович

В рамках предыдущего семестра было проведено углубленное изучение технологий параллельного программирования. Рассматривались три ключевых подхода:

MPI

Модель распределенной памяти и явного обмена сообщениями.

OpenMP

Директивная модель для систем с общей памятью.

Intel TBB

Задачный параллелизм и динамическая балансировка нагрузки.

Полученный опыт позволил выделить основные требования к инструментам автоматизации параллельных вычислений.

Анализ существующих технологий показал, что при разработке параллельных алгоритмов сохраняются повторяющиеся сложности:

- ❗ Необходимость ручного описания зависимостей;
- ❗ Управление потоками, синхронизацией и обменом данными;
- ❗ Зависимость реализации от конкретной вычислительной среды;
- ❗ Риск ошибок при работе с общей и распределенной памятью и коммуникациями.

Поэтому в текущем семестре работа была продолжена в направлении разработки собственного **task-based фреймворка** для выполнения графа задач.

На основе анализа существующих подходов были выделены ключевые проблемы, которые должен решать разрабатываемый инструмент:



Сложность управления

Избавление разработчика от ручного управления потоками и примитивами синхронизации.



Риски ошибок

Снижение риска Race Condition и Deadlock за счёт явного описания зависимостей и централизованного управления выполнением.



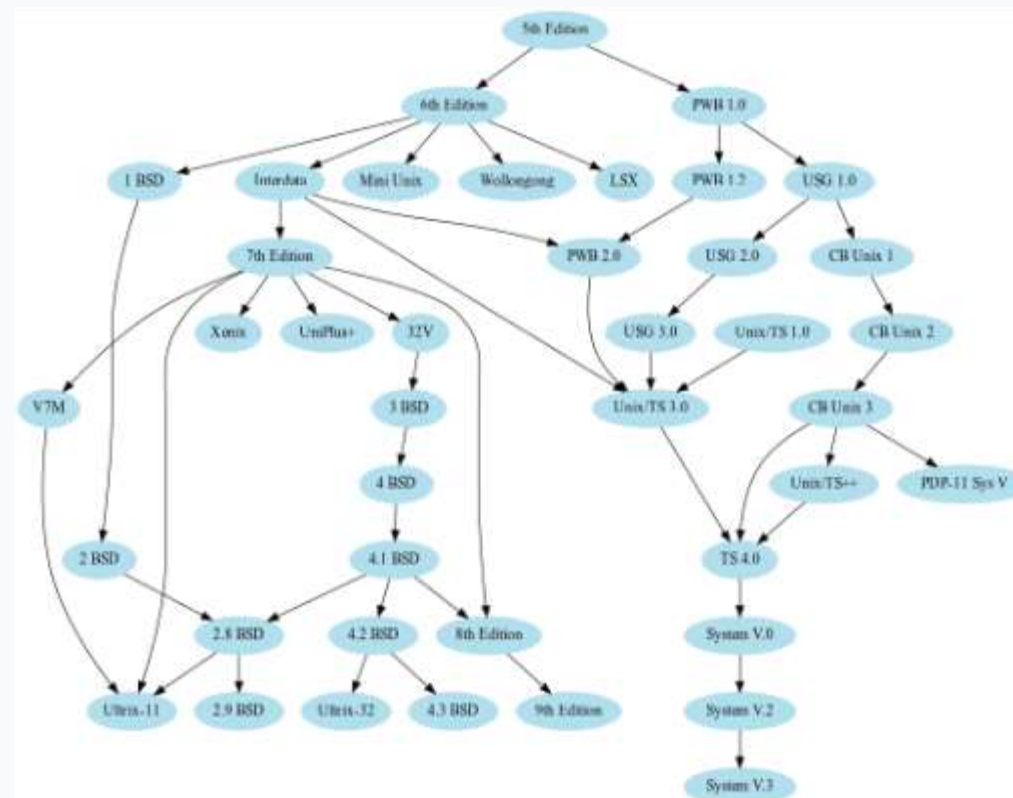
Масштабируемость

Адаптация одного алгоритма под разные среды (Threads/MPI) без изменения логики.

УНИВЕРСИТЕТ
ЛОБАЧЕВСКОГО
НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

- **Узлы (Nodes):** Вычислительные задачи — атомарные единицы работы (функторы или лямбда-выражения).

 **TaskContext:** Изолированное потокобезопасное хранилище для обмена данными между узлами.



Основная идея и принцип работы



Задачи + Зависимости

Ввод пользователя



Построение DAG

Внутренняя логика



Автоматическое исполнение

Результат работы

✓ Параллелизм «из коробки»

Фреймворк автоматически выявляет независимые задачи и запускает их параллельно на доступных ядрах процессора.

Taskflow

C++ библиотека для графов. Поддерживает вложенные подграфы, асинхронные задачи и профилирование.

CGraph

Фреймворк для DAG- и pipeline-сценариев. Особенности: условия, циклы и Python-интерфейс.

TaskTorrent

Runtime для общей и распределенной памяти. Ориентация на MPI и снижение накладных расходов.

Система	Основной акцент
Taskflow	Развитый C++ API, графы задач, профилирование
CGraph	DAG/pipeline-сценарии, условия, события, Python
TaskTorrent	Распределенный task-based runtime, MPI
Наш прототип	Минимальная архитектура DAG + Thread backend + задел под MPI

Цель: исследовать базовые механизмы выполнения DAG и оценить накладные расходы собственной архитектуры.



TaskGraph

Описание структуры:
регистрация задач,
определение
входных/выходных ключей
данных и связей между ними.



Executor

Диспетчеризация: обход графа,
мониторинг готовности задач
(счётчик зависимостей) и
запуск через драйвер.



Drivers

Абстракция исполнения:
ThreadsDriver (пул потоков
локально) или **MpiDriver**
(распределенно).

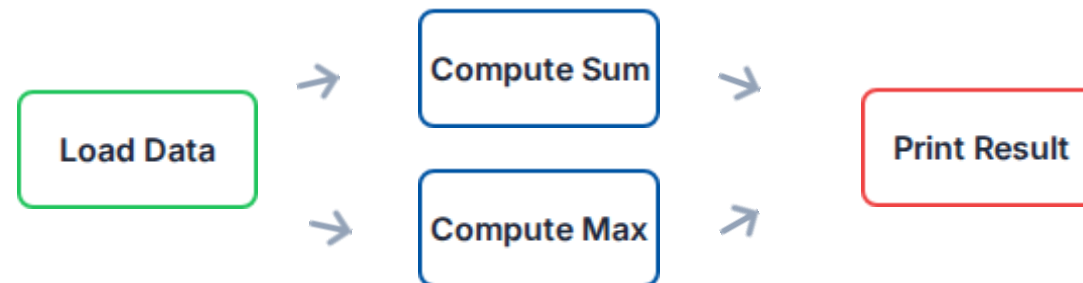
Схема выполнения графа



Thread Driver: Пул потоков (ThreadPool) для локальных многоядерных систем. **Статус: Готов.**

MPI Driver: Реализована сериализация TaskContext для распределенных вычислений. **Статус: В разработке.**

Пример выполнения графа



Механизм управления: Система автоматически определяет, что задачи *Sum* и *Max* независимы и могут выполняться параллельно в разных потоках. Задача *Print* будет заблокирована до завершения обоих предшественников.

Low Exchange Workload

Тестирование **параллельной суммы** массива.
Множество мелких независимых задач. Основная
цель: оценка накладных расходов планировщика при
высокой частоте запусков.

High Exchange Workload

Тестирование **конвейерной обработки** (8 стадий).
Передача тяжелых векторов данных между узлами.
Основная цель: оценка эффективности механизмов
копирования и передачи данных.

 Параметры тестов: 6 ядер CPU, 5 повторов для усреднения. Проверка корректности: **Diff = 0**.

Результаты: Low Exchange Workload

Сценарий: Параллельное суммирование массива чанками (минимальный обмен данными).

Размерность (N)	Manual Implementation (ms)	Framework Prototype (ms)	Slowdown (Factor)
100 000	0.355	2.250	6.344x
500 000	0.502	8.600	17.125x
1 000 000	0.864	13.674	15.836x
2 000 000	1.191	20.662	17.353x
5 000 000	2.009	61.678	30.705x

Анализ: На малых задачах накладные расходы (Overhead) фреймворка значительны. Причина: Использование `get_copy()` для каждого чанка создает избыточное копирование векторов в памяти.

Результаты: High Exchange Workload

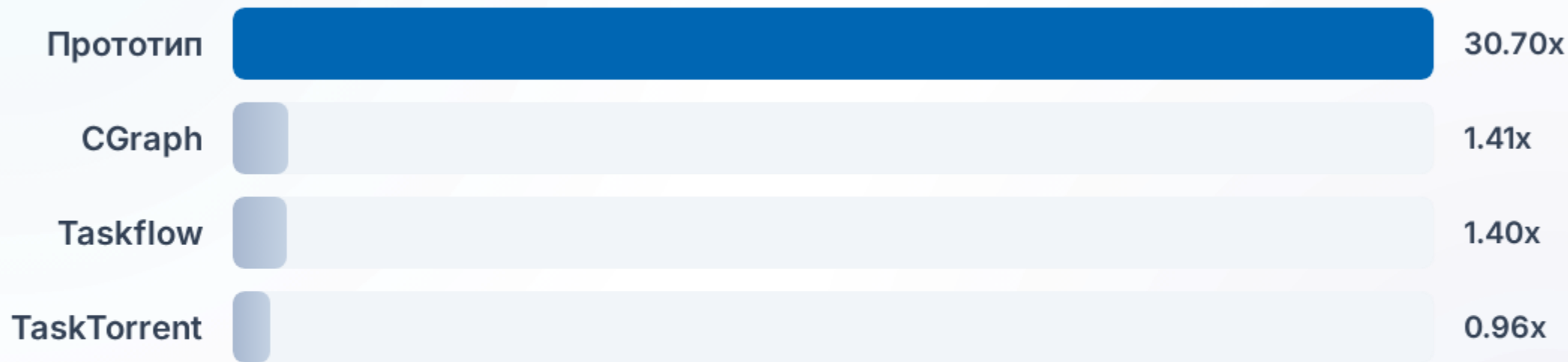
Сценарий: Конвейер обработки (8 стадий) с интенсивной передачей векторов.

Размерность (N)	Manual Implementation (ms)	Framework Prototype (ms)	Slowdown (Factor)
50 000	0.122	1.162	9.524x
100 000	0.253	2.174	8.603x
200 000	0.504	3.414	6.770x
500 000	1.953	7.934	4.063x
1 000 000	4.284	18.981	4.431x

Анализ: На "тяжелых" вычислительных задачах разрыв значительно сокращается. Фреймворк эффективно масштабируется, когда полезная работа превышает время диспетчеризации.

Сравнение с аналогами: Low Exchange

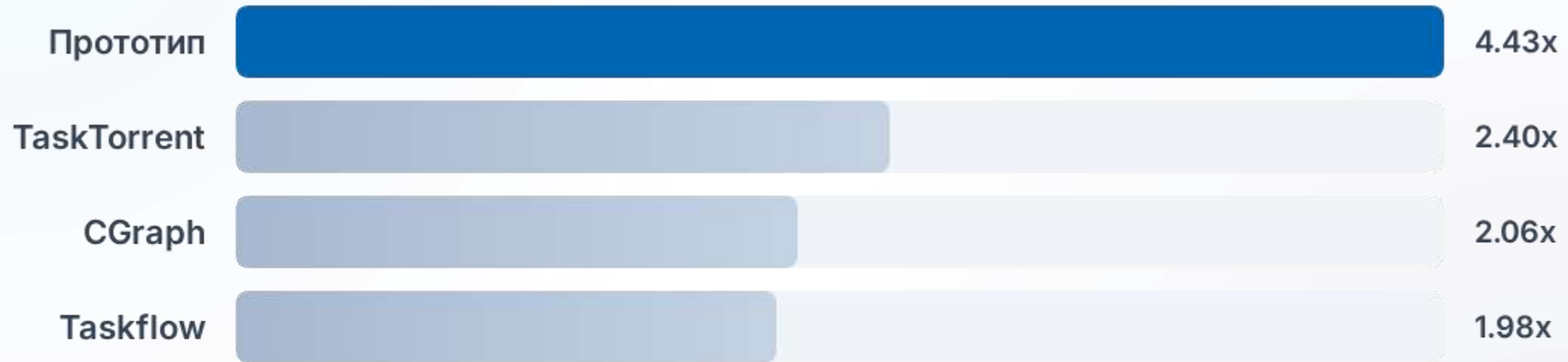
Показатель Slowdown (отставание от ручной реализации) при $N = 5\,000\,000$



Значительное отставание прототипа ожидаемо и обусловлено избыточным копированием данных, тогда как промышленные аналоги уже используют оптимизированную передачу по ссылке.

Сравнение с аналогами: High Exchange

Показатель Slowdown (отставание от ручной реализации) при $N = 1\,000\,000$



В конвейерных задачах отставание прототипа сильно сокращается: большое количество полезных вычислений перекрывает затраты на работу самого фреймворка.

Итоги работы

- Разработан прототип Task-based фреймворка на C++.
- Обеспечена абстракция от физического уровня исполнения.
- Корректность подтверждена: во всех тестах **Diff = 0**.

Дальнейшие шаги

- 🔗 **Zero-copy:** Переход на передачу ссылок для исключения копирования.
- 🔧 **MPI Driver:** Полноценная реализация распределенного режима.



1. Воеводин, В. В. Математические основы параллельных вычислений / В. В. Воеводин. — М. : Изд-во МГУ, 1991. — 304 с.
2. Воеводин, В. В. Параллельные вычисления / В. В. Воеводин, Вл. В. Воеводин. — СПб.: БХВ-Петербург, 2002. — 608 с.
3. Гергель, В. П. Теория и практика параллельных вычислений / В. П. Гергель. — М. :Интернет-Университет Информационных Технологий
4. Зыков, А. А. Введение в теорию графов / А. А. Зыков. — М. : Наука, 1987. — 384 с.
5. Foster, I. Designing and Building Parallel Programs: Concepts and Tools for SoftwareEngineering / I. Foster. — Reading, MA : Addison-Wesley, 1995.
6. Grama, A. Introduction to Parallel Computing / A. Grama, A. Gupta, G. Karypis, V. Kumar.— 2nd ed. — Harlow : Pearson, 2003.
7. Rauber, T. Parallel Programming: for Multicore and Cluster Systems / T. Rauber, G. Runger.— Berlin : Springer, 2013.